

What every machine is doing, second by second

A runtime trace of HADS, Burst-HADS and R-BurstHADS on one identical workload

This report follows a single simulated run from start to finish and reports, at every moment, the state of every virtual machine in the fleet: which ones are executing, which ones are holding queued work, which ones have been frozen by the cloud provider, how many CPU credits each burstable machine has left, and where the money is going. All three schedulers are given exactly the same tasks, the same starting fleet, the same deadline, and the same interruptions at exactly the same instants, so any difference in the pictures below is a difference in scheduling policy and nothing else.

The run being traced

One run was selected rather than an average, because averages hide mechanism. The point of this document is to show what actually happens, not how often it happens.

Setting	Value
Tasks in the bag	40 independent tasks, execution times drawn from 100 to 350 workload seconds
Deadline	289 seconds
Starting fleet	9 spot VMs, 1 burstable t3.large, 1 on-demand c5.large
Interruption scenario	k_h = 5, k_r = 0: hibernations arrive as a Poisson process, nothing ever resumes
Random seed	0, identical for all three schedulers

The interruptions land at 17.3, 20.9, 30.0, 31.5, 37.4, 41.4, 81.9, 88.5 and 107.5 seconds. Because no machine ever resumes, by t = 108 all nine of the original spot machines are frozen, and every task that was on them has to be somewhere else. Everything the three schedulers do differently is a response to those same nine moments.

Outcome	HADS	Burst-HADS	R-BurstHADS
Makespan	285.5 s	275.8 s	247.4 s
Reported cost	\$0.1915	\$0.2121	\$0.1374
Deadline misses	0 of 40	0 of 40	0 of 40
Machines ever created	17	20	18
Tasks rescued onto burstables	0	8	4

How to read the timeline charts

Each machine gets one row. Time runs left to right. Every row carries two thin lanes: the upper, dark green lane is work actually executing, and the number printed inside it is how many tasks are running in parallel on that machine at that moment. The lower, pale green lane is work that has been assigned to the machine but is still sitting in its queue waiting for a free core, and the number inside it is how many tasks are waiting. A machine with a four-core instance type can show a 4 in the upper lane; a single-core burstable never shows more than a 1.

A red hatched bar means the provider has frozen that machine. In this scenario nothing ever comes back, so once a row turns red it stays red to the end. A thin grey line means the machine is alive and paid for but holding nothing at

all. A blue triangle marks the moment a newly requested machine finishes booting and becomes usable. A row that has no marks at all before some point in time did not exist yet.

One detail worth stating because it affects how the charts were built. The simulator keeps a state label on each VM, but that label is only refreshed when a task completes, so it reads "idle" for a machine that is in fact executing. The charts therefore derive occupancy from the machine's actual running and queued task lists, which is what the rest of the simulator treats as ground truth. This is a reporting detail, not a scheduling defect: every consumer of that state label in the scheduling code only ever tests it for "frozen" or "terminated", both of which it does track correctly, and the one place that could have misfired, the idle-machine shutdown timer, is explicitly guarded against it.

HADS: pay full price, and never touch the burstable

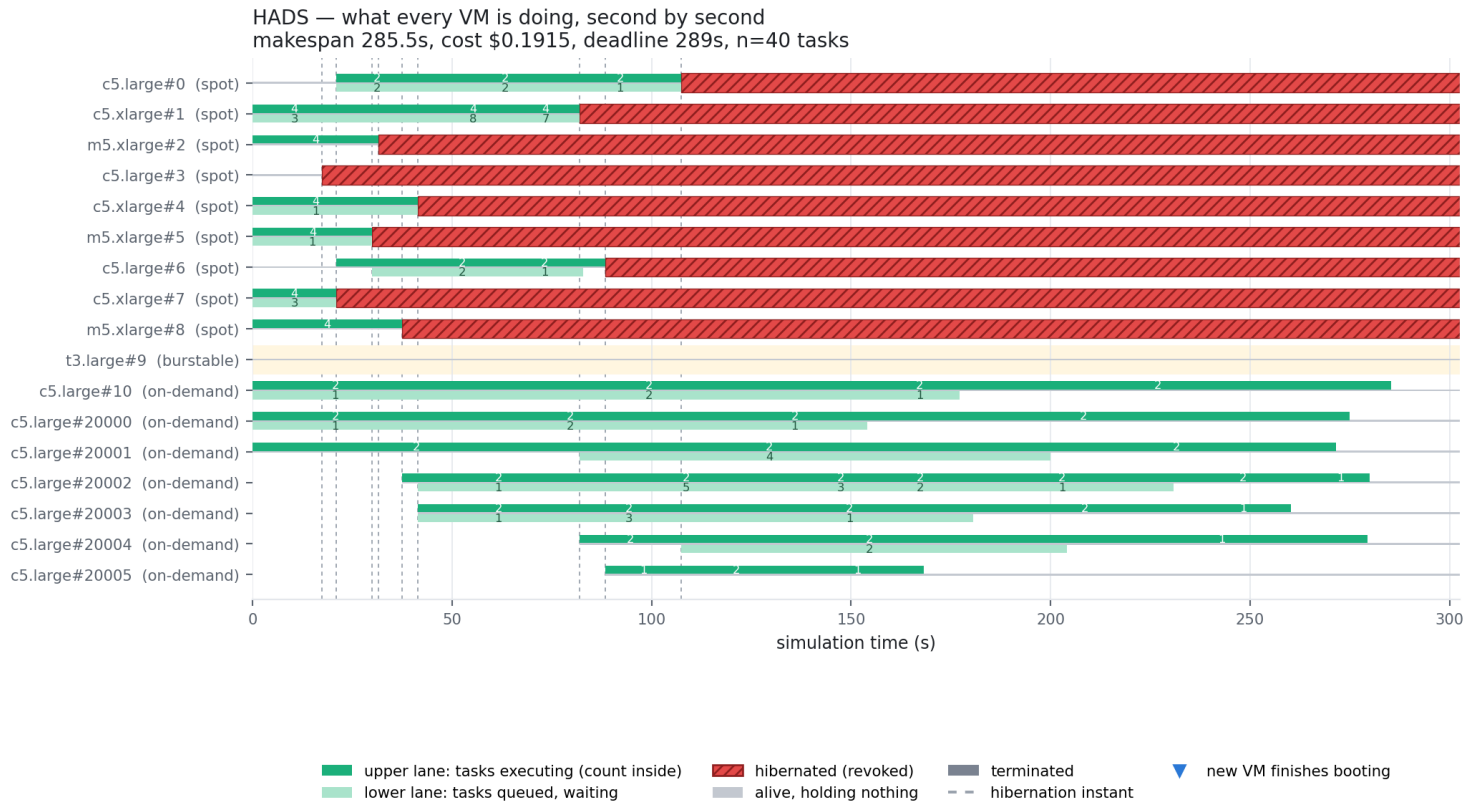


Figure 1. HADS. The t3.large row is empty for the entire run.

The first thing to notice is the row labelled t3.large#9. It is blank from start to finish. HADS has a burstable machine sitting in its fleet the whole time and never sends it a single task. This is correct behaviour, not a bug: rescuing work onto burstable instances is the contribution Burst-HADS added, and HADS predates it. The blank row is the baseline this whole line of work exists to improve on.

What HADS does instead is visible in the bottom six rows. Every time a spot machine freezes, HADS walks its recovery ladder: try another machine already in the fleet, then an on-demand machine already in the fleet, and if neither can meet the deadline, buy a brand new on-demand machine. Across the nine interruptions it buys six new on-demand machines, numbered 20000 through 20005. On-demand is the most expensive thing on the price list at \$0.085 per hour for two cores, roughly 2.8 times the per-core price of the c5.xlarge spot instances the workload started on.

Follow one machine to see the pattern. c5.xlarge#1 is executing four tasks in parallel with eight more queued behind them when it freezes at $t = 81.9$. That is eleven pieces of unfinished work needing a home in a fleet that has already lost five machines. HADS spreads them across four different on-demand machines in a single burst of activity at that instant. By $t = 108$ the entire original spot fleet is gone and the last 175 seconds of the run are carried almost entirely by rented on-demand capacity.

Burst-HADS: the burstable machines finally get used

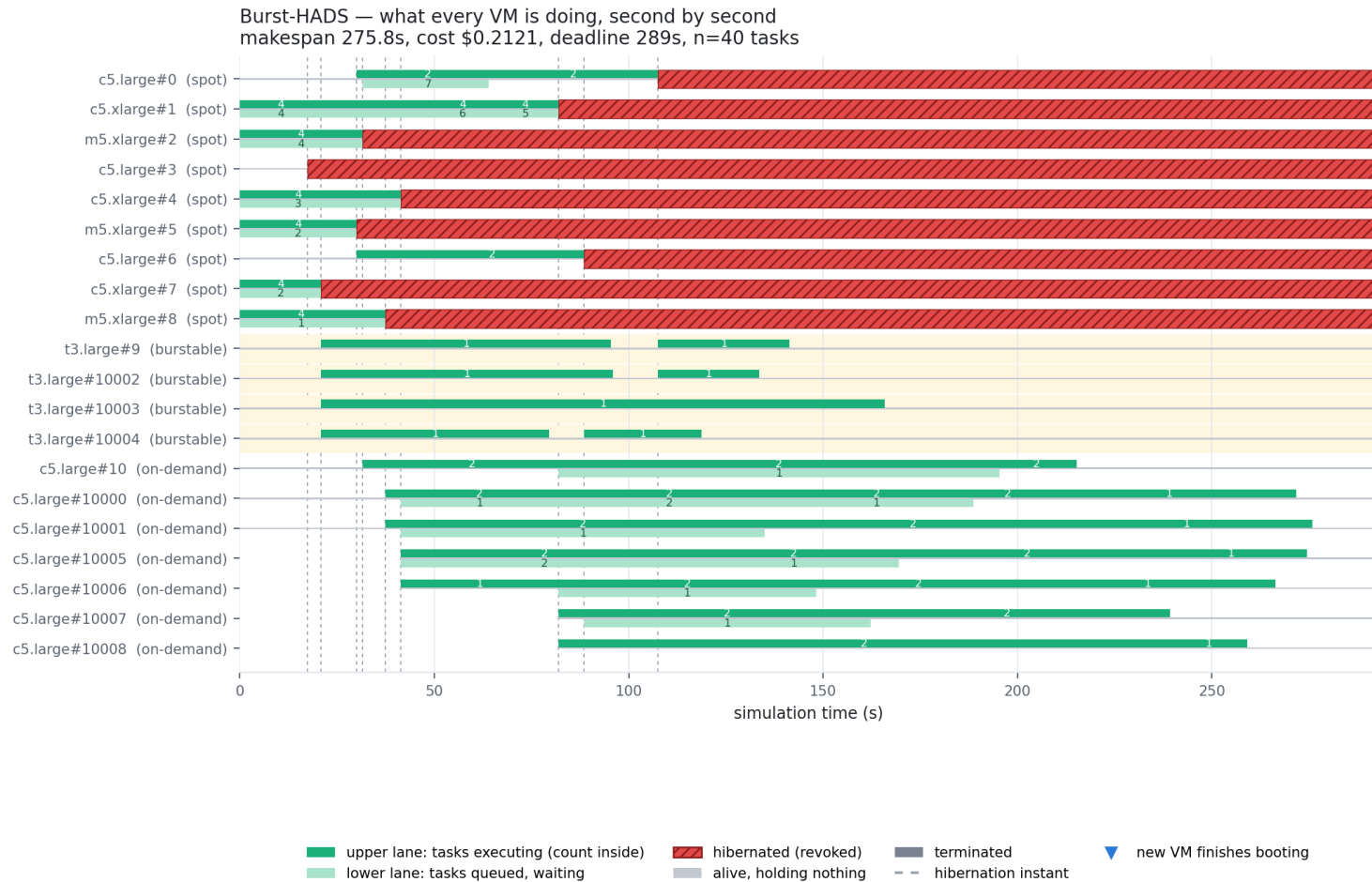


Figure 2. Burst-HADS. Four burstable rows are now busy, but nine on-demand machines are still bought.

At $t = 20.9$, when `c5.xlarge#7` freezes holding six unfinished tasks, Burst-HADS does something HADS cannot. It reaches for burstable capacity first. Task 32 goes to `t3.large#9`, the burstable that was already in the fleet, and three further burstable machines are drawn from the reserve pool to take tasks 16, 5 and 17. Only the last two tasks fall through to an ordinary spot machine.

Each of those burstable machines takes exactly one task and no more. That is not an implementation shortcut, it is the published rule. The burst migration procedure selects its candidates from the set of idle burstable machines and moves each one out of that set the moment it receives work, so a burstable machine holds one task at a time whether it is running in baseline mode or in burst mode. The timeline confirms it: no burstable row ever shows a number above 1 in either lane.

Burst-HADS still buys on-demand machines, nine of them, more than HADS. This is the honest shape of the trade-off in the published design. Burstable rescue shortens the makespan, from 285.5 to 275.8 seconds here, because a rescued task starts running immediately at full speed instead of queueing behind other work. But the burstable pool is small and each machine takes only one task, so the moment more than a handful of tasks need rescuing at once, the overflow goes to on-demand exactly as it did in HADS. The reported cost goes up rather than down.

R-BurstHADS: buy replacement capacity instead of renting rescue capacity



Figure 3. R-BurstHADS. Two new spot machines, #100 and #101, are requested and absorb most of the displaced work.

R-BurstHADS uses the burstable machines the same way Burst-HADS does, and at the same instant: at $t = 20.9$ tasks 32, 16, 5 and 17 land on the same four burstable machines. The difference appears at $t = 41.4$, when `c5.xlarge#4` freezes holding twelve unfinished tasks. Instead of scattering them across newly rented on-demand machines, R-BurstHADS requests one new spot machine, `c5.xlarge#100`, and queues eleven of the twelve tasks on it. A second, `c5.xlarge#101`, is requested at $t = 81.9$.

The blue triangles show the cost of that decision. Machine #100 is requested at $t = 41.4$ but only becomes usable at $t = 86.3$, and #101 is requested at $t = 81.9$ and usable at $t = 126.9$. Those eleven tasks sit in a queue for forty-five seconds before anything runs. This is why the provisioning decision is guarded by an explicit deadline test: the scheduler only takes this route when the remaining slack is more than twice the startup latency, and falls back to instant on-demand rescue when it is not. Two on-demand machines, #10000 and #10001, are bought here for exactly that reason, against six for HADS and nine for Burst-HADS.

The payoff is visible in the last third of the chart. Where HADS finishes on six rented on-demand machines and Burst-HADS on nine, R-BurstHADS finishes on two four-core spot machines it bought at spot prices, and it finishes 38 seconds earlier.

One thing this trace shows that is not flattering

The last task in the R-BurstHADS run finishes at $t = 247.4$ on `t3.large#10002`, a single-core burstable machine, and it took 151 seconds to run there. That task was not rescued onto the burstable in burst mode; it was picked up by the work-stealing procedure, which places stolen work on a burstable machine in baseline mode, where the machine runs

at twenty percent of full speed and earns credits instead of spending them. The credit trace confirms it: that machine's balance goes up by 1.5 while running the task, not down.

So on this seed the makespan of R-BurstHADS is set by a task deliberately placed on the slowest machine in the fleet. That is faithful to the published work-stealing rule, and R-BurstHADS still wins by a wide margin, but it is a clear place where the design could be improved: stealing work onto a baseline-mode burstable should be checked against the remaining slack the same way a burst-mode rescue already is.

The fleet as a whole

Fleet composition over time — the red band at the bottom is how many machines are frozen

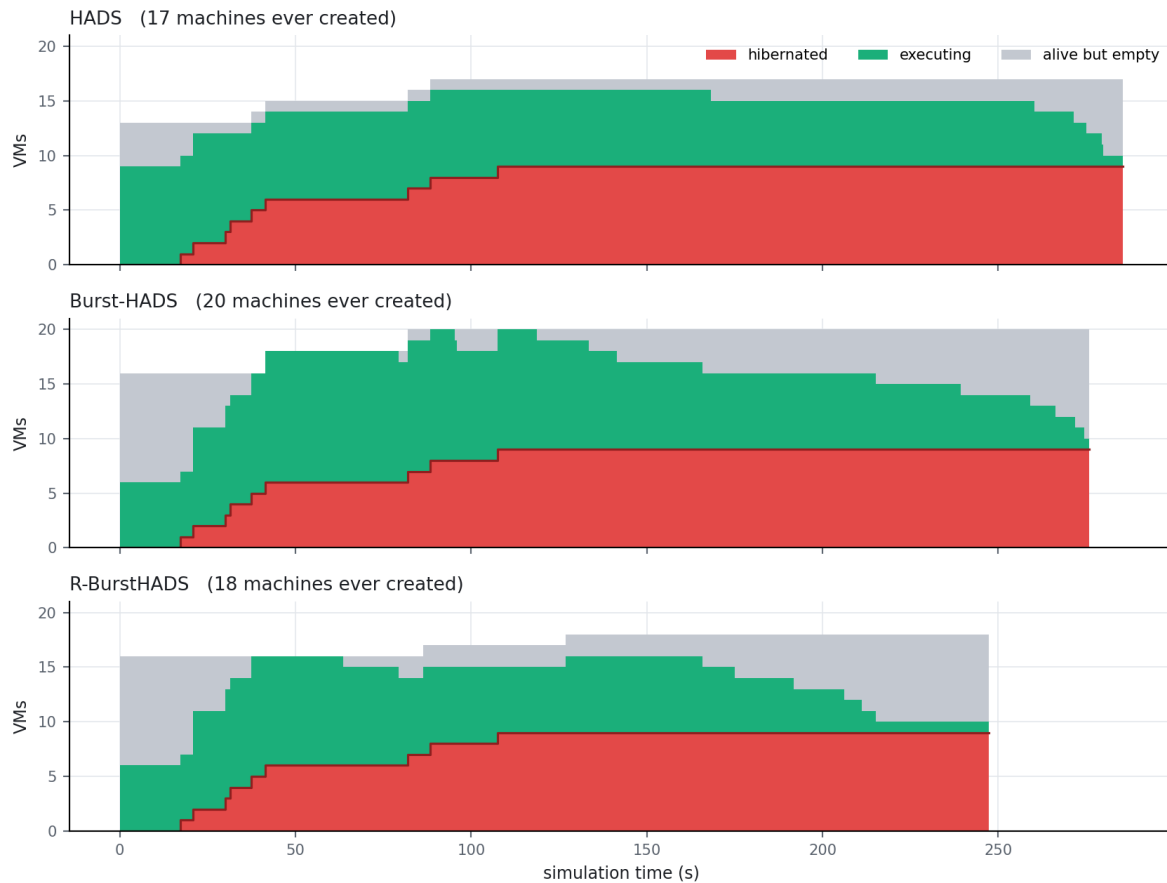


Figure 4. How many machines are frozen, executing, or alive but empty, at every moment.

Read the red band at the bottom of each panel: that is capacity the provider has taken away, and it grows to nine machines in every panel because all three schedulers face identical interruptions. The green band above it is capacity actually doing work. The grey band on top is capacity that is alive, being paid for, and holding nothing.

The interesting comparison is the grey. All three schedulers end their run with a growing band of paid-for idle machines, and R-BurstHADS reaches that state soonest because it finishes soonest. Finishing early does not, in the current model, stop the meter. That leads directly to the next section.

Where the money actually goes

Where the money goes — and how much of it is spent after the work is already done

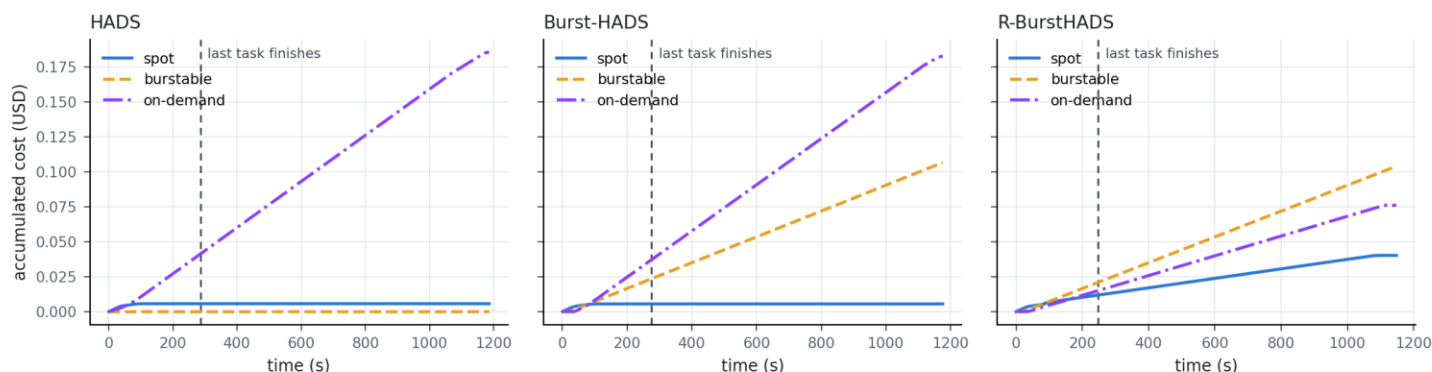


Figure 5. Accumulated cost per market. The dashed line is the moment the last task finishes.

Up to the dashed line, the three schedulers cost almost the same: \$0.047 for HADS, \$0.066 for Burst-HADS, \$0.048 for R-BurstHADS. Everything to the right of the dashed line is machines being billed after the work is already done. In this run that tail is 900 seconds long, because an idle machine is only shut down when its 900-second allocation cycle expires, and it accounts for roughly three quarters of every reported cost figure.

The composition of the spending is the substantive result. HADS spends almost everything on on-demand capacity and nothing at all on burstable. Burst-HADS adds burstable spending on top of a barely reduced on-demand bill. R-BurstHADS is the only one of the three whose largest single line item is not on-demand.

It is worth being precise about why the burstable machines are not a cost saving in themselves. A t3.large is priced at \$0.0832 per hour for two cores; the on-demand c5.large is \$0.0850 per hour for two cores. Per core-second the burstable is about two percent cheaper, which is nothing. The burstable instance is a latency instrument, not a cost instrument: what it buys is a machine that is already running and can absorb a rescued task at full speed immediately. The cost advantage in R-BurstHADS comes from somewhere else entirely, from replacing on-demand capacity with fresh spot capacity at roughly a third of the per-core price.

The burstable credit balance, machine by machine

Burstable CPU credits — every drop is one rescued task run at full speed in burst mode

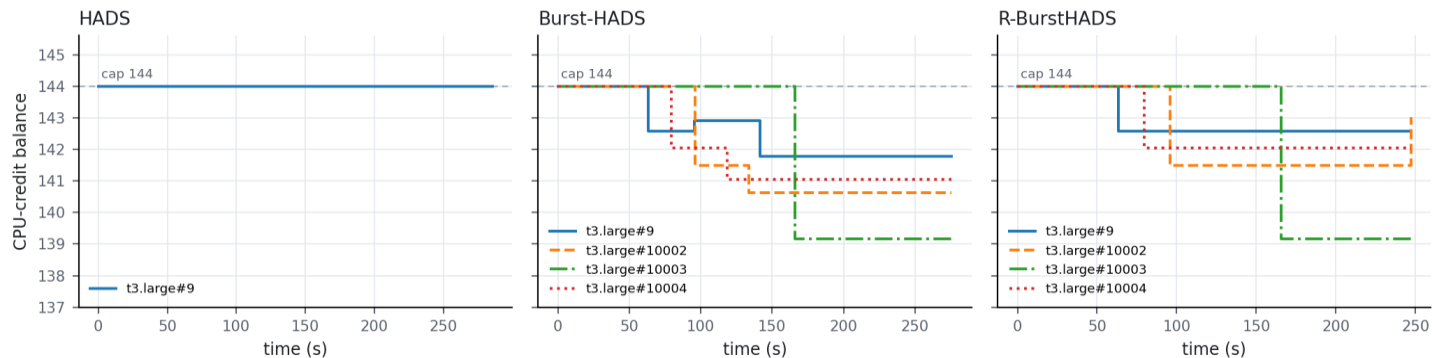


Figure 6. CPU-credit balance of every burstable machine over the run.

Every burstable machine starts at the cap, 144 credits. A machine running a task in burst mode spends credits at a rate of one per minute per core, so a task that occupies it for ninety seconds costs 1.5 credits. A machine running in baseline mode, or sitting idle, earns credits at 36 per hour up to the same cap. In the simulator the balance is settled in one lump when a task completes, which is why the lines are staircases rather than ramps.

Quantity	Value
Starting balance	144 credits, the t3.large cap
Earn rate	36 credits per hour, i.e. 0.01 per second
Spend rate in burst mode	1 credit per minute per core
Credits needed for a task	execution seconds divided by 60, times the machine speed
Largest drawdown observed	4.8 credits, on t3.large#10003 running task 5
Lowest balance reached	139.2 of 144, about 3 percent of the budget

The HADS panel is a flat line at 144. It never spends a credit because it never uses the machine. The other two panels are nearly identical to each other, which is expected: R-BurstHADS inherits the burst migration procedure unchanged and only differs in what it does after the burstable pool is exhausted.

The honest reading of this figure is that at this workload size the credit budget is not binding and is nowhere near binding. The deepest any machine goes is 4.8 credits out of 144. Credits are modelled correctly and they do get spent, but a reader would be entitled to ask whether the credit mechanism is doing any real work in these experiments. At forty tasks it is not. It would begin to bite only with substantially longer tasks, many more consecutive rescues onto the same machine, or a burstable pool small enough that the same machines are reused repeatedly. That is a fair limitation to state rather than to leave for a reviewer to find.

One task, followed end to end

The question this report was asked to answer is what it looks like when a task is placed on a spot machine, that machine is frozen, the task moves to a burstable machine, and the burstable machine spends credits to finish it in time. Task 5 in the R-BurstHADS run is exactly that story.

Time	What happens to task 5
t = 0	Placed by the initial solution on c5.xlarge#7, a four-core spot machine, alongside three other running tasks and two queued behind them.
t = 0 to 20.9	Executing. A checkpoint is maintained continuously so that progress is not lost if the machine is taken away.
t = 20.9	The provider freezes c5.xlarge#7. Billing on that machine stops at this instant. Six unfinished tasks need a new home, task 5 among them, with 264 seconds of work still remaining.
t = 20.9	The burst migration procedure runs. It looks first at idle burstable machines, finds t3.large#10003 available, checks that the machine has the memory and that the task can still finish before the 289-second deadline there, and moves it. The machine is removed from the idle burstable set so nothing else can be placed on it.
t = 20.9	The task starts immediately in burst mode, at the machine's full speed of 2 rather than its baseline 0.4, with the ten percent checkpointing overhead applied.
t = 165.8	The task completes, 123.9 seconds before the deadline. The credit ledger is settled: 4.8 credits are deducted, taking the balance from 144.0 to 139.2.

For contrast, the same task in the HADS run is rescued at the same instant onto c5.xlarge#4, an ordinary spot machine that is already executing four tasks and holding a queue. It waits for a free core, and c5.xlarge#4 is itself frozen at t = 41.4 before the task has finished, so it must be moved a second time, to a newly purchased on-demand machine. One interruption, two migrations, and a machine bought at the highest price on the list.

A defect this trace exposed in the cost measurement

This section is the most important part of the document and it does not reflect well on the current results. Building the spending chart required reading each machine's billing record directly, and doing so revealed that the cost metric applies three different billing rules within a single number.

Kind of machine	How it is currently billed
Frozen by the provider	Billed up to the moment it was frozen. Correct.
Shut down by the idle timer	Billed up to the shutdown, which is 900 seconds after it last held work, and therefore well past the end of the job.
Still alive at the end	Billed only up to the makespan, because the metric caps open billing intervals there.

The intent of the code is clearly to stop the meter at job completion, and for machines still running at the end it does. For machines the idle timer shut down it silently does not, because the metric only caps intervals that are still open. The consequence is that the reported cost of a run depends heavily on how many machines happen to be left holding a 900-second idle tail, and burstable machines are exempt from the idle timer so they are consistently billed under the stricter rule while everything else is billed under the looser one.

To size this, cost was recomputed from the same simulations under three consistent policies. Policy A is what is currently reported. Policy B truncates every machine's bill at the makespan, which asks what the job itself cost. Policy C bills every machine to its real shutdown, which is what a cloud provider would actually invoice. Costs below are relative to HADS in the same cell, so a negative number means cheaper than HADS.

Configuration	Scheduler	A, as reported	B, to makespan	C, to shutdown
n=40, k_h=5, k_r=0	Burst-HADS	+10.7%	+41.2%	+54.2%
	R-BurstHADS	-28.3%	+1.7%	+13.6%
n=300, k_h=5, k_r=0	Burst-HADS	+4.9%	+15.2%	+15.9%
	R-BurstHADS	-66.1%	-56.3%	-61.2%
n=300, k_h=1, k_r=0	Burst-HADS	+59.5%	+119.6%	+69.1%
	R-BurstHADS	+45.4%	+30.7%	+66.1%
n=300, k_h=5, k_r=5	Burst-HADS	+12.6%	+39.9%	+39.5%
	R-BurstHADS	+2.5%	-22.5%	+32.9%

Three things follow from this table, and they should be stated plainly.

First, the makespan results are untouched. Billing policy has no effect on when tasks finish, and the makespan advantage is the stronger of the two claims in any case.

Second, at the small trace scale the billing tail dominates completely. The apparent 28 percent cost advantage at forty tasks disappears under either consistent policy. Nothing in this document's cost comparison at $n = 40$ should be quoted as a result; it is included only to explain the mechanism.

Third, at the scale the experiments actually run, the headline cost advantage survives in the scenario the paper leads with, and it survives comfortably: 56 to 66 percent cheaper than HADS at three hundred tasks under all three policies. But the individual seeds shown here also change by tens of percentage points between policies, and in one of the four

cells the sign of the R-BurstHADS result flips. The published cost numbers cannot be defended until the billing rule is made consistent and the full thirty-seed sweep is re-run against it.

Recommendation

Adopt policy C. Bill every machine from launch to the moment it is genuinely gone, whether that is hibernation, idle shutdown, or the end of the simulation, and remove the makespan cap entirely. It is the only one of the three that corresponds to what a customer is charged, it treats burstable and non-burstable machines the same way, and it is the least favourable to the proposed scheduler, which makes the result harder to argue with rather than easier.

Adopting it exposes a second, genuine weakness that the current metric hides. Under policy C a scheduler is charged for finishing early and then leaving its fleet running, and R-BurstHADS does exactly that: in the $k_h = 5$, $k_r = 5$ case it finishes in 798 seconds against HADS's 1583, then idles a large fleet for another 1887 seconds. Releasing provisioned machines as soon as the bag of tasks is empty, rather than waiting out a 900-second allocation cycle, is a small change that would convert the existing makespan advantage into a cost advantage instead of paying it back.

All figures in this document were generated from `experiments/trace_out/`, which records a snapshot of every machine at every event in the three runs. The billing comparison was produced by `experiments/billing_audit.py`, which re-reads the billing intervals of a completed simulation and does not modify the simulator.